



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5

по дисциплине «Прикладные задачи математической статистики»

Студент группы *ИНБО-20-23. Школьный И.В.*

(подпись)

Преподаватель *Трушина В.И.*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

ПОСТАНОВКА ЗАДАЧИ.....	3
ВЫПОЛНЕНИЕ РАБОТЫ	5
1.1 Файл 1	5
1.2 Файл 2	10
1.3 Файл 3	12
ВЫВОД.....	15

ПОСТАНОВКА ЗАДАЧИ

Цель работы: освоить методы регрессионного анализа, включая построение линейных регрессионных моделей, оценку их адекватности, проверку значимости параметров, анализ гетероскедастичности и применение теста Чоу для сравнения моделей на разных подвыборках.

Постановка задачи.

Для первого файла выполнить:

1. Оценить коэффициент корреляции Пирсона $r(x,y)$ между переменными x и y .
2. По шкале Чеддока охарактеризовать силу корреляционной связи.
3. Проверить статистическую значимость коэффициента корреляции Пирсона с использованием t -критерия.
4. Построить линейную регрессионную модель, оценить коэффициенты a и b .
5. Проверить адекватность модели с помощью F -критерия Фишера.
6. Оценить значимость коэффициентов a и b модели с использованием t -критерия.
7. Построить доверительные интервалы для коэффициентов a и b .
8. Оценить интервал прогноза для линейной модели на $\Delta x=3$ вперёд.

Для второго файла выполнить:

1. С помощью теста Чоу проверить необходимость разделения выборки по каждой из качественных факторных переменных.
2. В случае значимости разделения — построить две линейные регрессионные модели для каждой подвыборки и оценить их коэффициенты.

Для третьего файла выполнить:

1. Построить линейную регрессионную модель, оценить коэффициенты a и b .

2. Проверить значимость коэффициентов и адекватность модели.
3. Двумя методами (тест Спирмена и тест Гольдфельда-Квандта) проверить наличие гетероскедастичности в данных.

Уровень значимости для всех статистических тестов $\alpha = 0.05$. В данной практической работе выполняется 28 вариант.

ВЫПОЛНЕНИЕ РАБОТЫ

1.1 Файл 1

Первый файл содержит 2 ряда данных. Первый столбец x содержит факторную переменную, второй столбец y — результирующую. С помощью библиотеки `numpy` считаем данные из файла и оценим коэффициент корреляции Пирсона $r(x, y)$ между двумя переменными в первом и втором столбце. Код для вычисления корреляции Пирсона представлен в Листинге 1.

Листинг 1 — Вычисление коэффициента корреляции Пирсона

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
import warnings

def pearson_corr(x, y):
    return np.corrcoef(x, y)[0, 1]

data = np.loadtxt('1.txt', delimiter=',')
x, y = data[:, 0], data[:, 1]
n = len(x)
r = pearson_corr(x, y)
print(f"Коэффициент корреляции Пирсона: r = {r:.4f}")
```

Запустим скрипт и получим, что связь между переменными положительная, поскольку коэффициент корреляции Пирсона = 0.9858.

Согласно шкале Чеддока корреляционная связь между величинами весьма высокая.

Проверим статистическую значимость коэффициента корреляции Пирсона с помощью t -статистики. Код для проверки представлен в Листинге 2.

Листинг 2 — Проверка статистической значимости коэффициента корреляции

```
def t_test_corr(r, n):
    t = abs(r) * np.sqrt((n - 2) / (1 - r**2))
```

Продолжение Листинга 2

```
t_critical = stats.t.ppf(1 - alpha/2, n-2)
return t, t_critical, t > t_critical
t_stat, t_crit, significant = t_test_corr(r, n)
print(f"t-статистика = {t_stat:.4f}, критическое значение = {t_crit:.4f}")
print(f"Корреляция {'значима' if significant else 'не значима'})")
```

Запустим скрипт и получим следующие значения:

- t-статистика = 58.0724;
- критическое значение = 1.9845.

Поскольку значение t-статистики больше критического, корреляция можно считать значимой.

После установления значимости корреляции, построим линейную регрессию между столбцами и оценим значение коэффициентов линейной зависимости. Код для реализации данных пунктов представлен в Листинге 3.

Листинг 3 — Построение модели линейной регрессии

```
def linreg_coef(x, y):
    n = len(x)
    x_mean = np.mean(x)
    y_mean = np.mean(y)
    a = (np.sum(x*y) - n*x_mean*y_mean) / (np.sum(x**2) - n*x_mean**2)
    b = y_mean - a*x_mean
    return a, b

a, b = linreg_coef(x, y)
print(f"Линейная регрессия: y = {a:.4f}x + {b:.4f}")
```

Запустим скрипт и получим следующую формулу линейной регрессии (Формула 1).

$$y = 1.2982x + 2.4050. \quad (1)$$

Как видно из Формулы 1, коэффициент при переменной x имеет положительное значение, поскольку связь между переменными положительна.

График парной зависимости результирующей переменной от объясняющей переменной представлен на Рисунке 1.

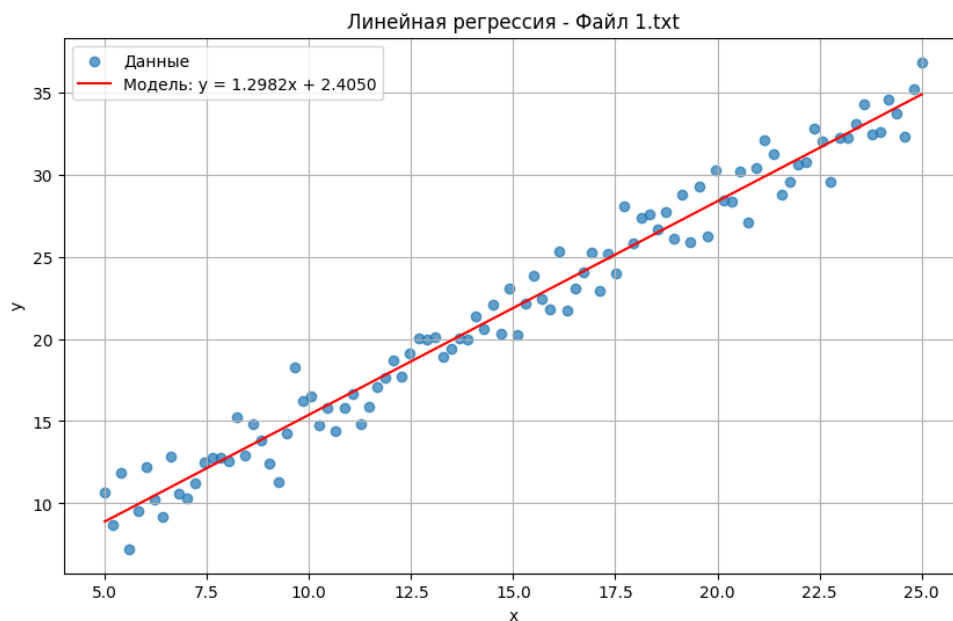


Рисунок 1 — График линейной регрессии

После построения модели линейной регрессии оценим ее адекватность с использованием критерия Фишера. Код для расчета статистики представлен в Листинге 4.

Листинг 4 — Вычисление статистики Фишера

```
def f_test_adequacy(x, y, a, b):
    n = len(x)
    y_pred = a*x + b
    S_mod2 = np.sum((y - y_pred)**2) / (n - 2)
    S_total2 = np.sum((y - np.mean(y))**2) / (n - 1)
    F = max(S_mod2, S_total2) / min(S_mod2, S_total2)
    F_critical = stats.f.ppf(1 - alpha, n-2, n-1)
    return F, F_critical, F > F_critical

F, F_crit, adequate = f_test_adequacy(x, y, a, b)
print(f"F-статистика = {F:.4f}, критическое значение = {F_crit:.4f}")
print(f"Модель {'адекватна' if adequate else 'не адекватна'})"
```

Запустим скрипт и получим следующие значения:

- F-статистика = 35.0546;
- критическое значение = 1.3950.

Поскольку значение F-статистики больше критического, модель можно считать адекватной.

После подтверждения адекватности модели оценим значимость полученных коэффициентов линейной регрессии. Сделаем это с помощью t-статистики. Код для расчета представлен в Листинге 5.

Листинг 5 — Оценка значимости полученных коэффициентов линейной регрессии

```
def significance_coefs(x, y, a, b, S_mod):
    n = len(x)
    sigma_x = np.std(x, ddof=1)
    m_a = S_mod / (sigma_x * np.sqrt(n))
    m_b = (S_mod * np.sqrt(np.sum(x**2))) / (sigma_x * n)

    T_a = a / m_a
    T_b = b / m_b
    t_critical = stats.t.ppf(1 - alpha/2, n-2)
    return (T_a, T_b, t_critical,
            abs(T_a) > t_critical, abs(T_b) > t_critical,
            m_a, m_b)

S_mod = np.sqrt(np.sum((y - (a*x + b))**2) / (n - 2))
T_a, T_b, t_crit, a_sig, b_sig, m_a, m_b = significance_coefs(x, y, a, b, S_mod)
print(f"Коэффициент a: t = {T_a:.4f}, {'значим' if a_sig else 'не значим'}")
print(f"Коэффициент b: t = {T_b:.4f}, {'значим' if b_sig else 'не значим'}")
```

Запустим скрипт и получим следующие значения:

- коэффициент a: $t = 58.3650$;
- коэффициент b: $t = 6.7185$.

Поскольку коэффициент t в обоих случаях больше критического, оба коэффициента линейной регрессии можно считать значимыми.

Построим доверительные интервалы для полученных коэффициентов. Код для реализации представлен в Листинге 6.

Листинг 6 — Построение доверительных интервалов

```
def conf_intervals(a, b, m_a, m_b, n):
    t_val = stats.t.ppf(1 - alpha/2, n-2)
    a_ci = (a - t_val * m_a, a + t_val * m_a)
    b_ci = (b - t_val * m_b, b + t_val * m_b)
    return a_ci, b_ci

a_ci, b_ci = conf_intervals(a, b, m_a, m_b, n)
print(f"a: [{a_ci[0]:.4f}, {a_ci[1]:.4f}]")
print(f"b: [{b_ci[0]:.4f}, {b_ci[1]:.4f}]")
```

Запустим скрипт и получим следующие интервалы для коэффициентов линейной регрессии:

- a: [1.2541, 1.3423];
- b: [1.6946, 3.1154].

После расчета доверительных интервалов изобразим модели, отвечающие граничным значениям полученных интервалов (Рисунок 2).

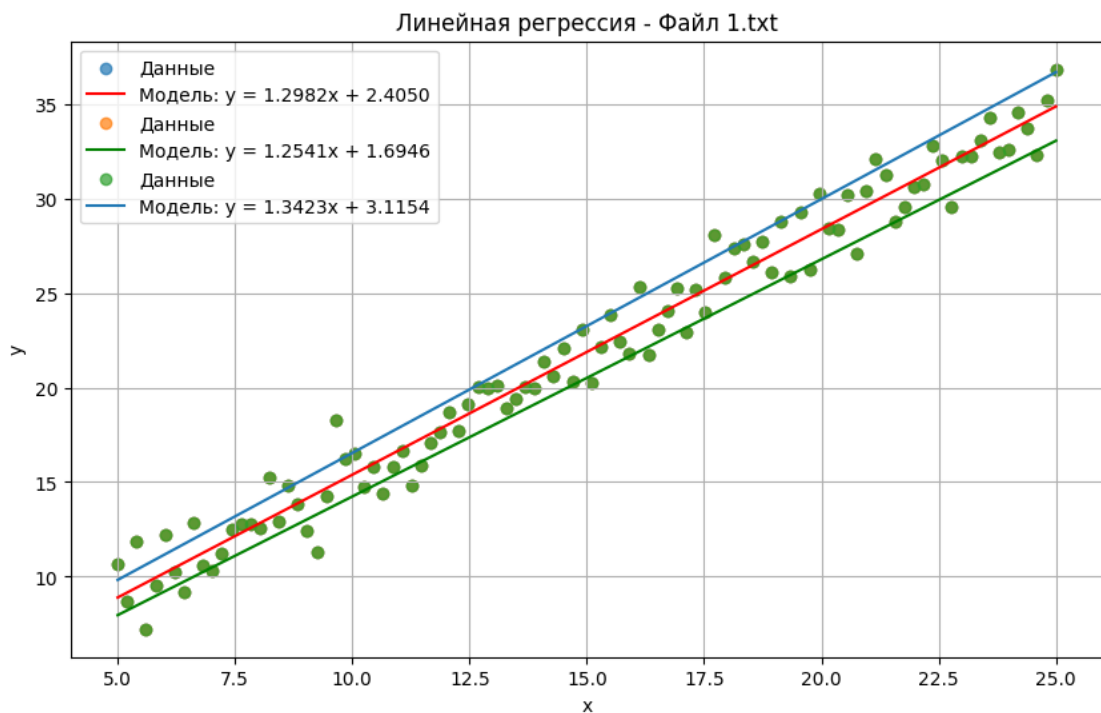


Рисунок 2 — Графики моделей, отвечающих граничным значениям доверительных интервалов

Теперь оценим интервал прогноза для линейной модели на $\Delta x = 3$ значения вперед. Интервал предсказания (доверительный интервал прогноза) – интервал, который с заданной вероятностью $(1 - \alpha)$ содержит фактическое значение целевой переменной y . Если прогнозное значение факторного признака x подставить в уравнение регрессии, мы получаем для него предсказанное значение y , которое является точечной оценкой для фактического y .

Код для расчета интервала прогноза представлен в Листинге 7.

Листинг 7 — Расчет интервала прогноза

```
def pred_interval(x, y, a, b, delta_x=3):
    n = len(x)
    x_new = np.max(x) + delta_x
    y_pred = a * x_new + b
    S_mod = np.sqrt(np.sum((y - (a*x + b))**2) / (n - 2))
    sigma_x = np.std(x, ddof=1)
    x_mean = np.mean(x)
    t_val = stats.t.ppf(1 - alpha/2, n-2)
    E = t_val * S_mod * np.sqrt(1 + 1/n + (x_new - x_mean)**2 / (n * sigma_x**2))
    return x_new, y_pred, (y_pred - E, y_pred + E)

x_new, y_pred, pred_ci = pred_interval(x, y, a, b, delta_x=3)
print(f"Интервал прогноза для x = {x_new:.2f}:")
print(f"Точечный прогноз: {y_pred:.4f}")
print(f"Интервал: [{pred_ci[0]:.4f}, {pred_ci[1]:.4f}]")
```

Запустим скрипт и получим интервал прогноза для $x = 28.00$:

- точечный прогноз: 38.7546;
- интервал: [36.0921, 41.4171].

1.2 Файл 2

Второй файл содержит 4 ряда данных. Первый ряд (столбец) содержит количественную факторную переменную, следующие два — качественную факторную переменную, последний — результирующую переменную.

С помощью библиотеки `pymru` считаем данные, а затем с помощью теста Чоу обоснуем необходимость деления выборки по одной из качественных факторных переменных. Код для проведения теста Чоу представлен в Листинге 8.

Листинг 8 — Чоу тест

```
def chow_test(x, y, d):
    a_total, b_total = linreg_coef(x, y)
    y_pred_total = a_total * x + b_total
    RSS = np.sum((y - y_pred_total)**2)
    x1, y1 = x[d == 0], y[d == 0]
    x2, y2 = x[d == 1], y[d == 1]
    a1, b1 = linreg_coef(x1, y1)
    a2, b2 = linreg_coef(x2, y2)
    y_pred1 = a1 * x1 + b1
    y_pred2 = a2 * x2 + b2
    RSS1 = np.sum((y1 - y_pred1)**2)
    RSS2 = np.sum((y2 - y_pred2)**2)

    k = 2
    n = len(x)
    F = ((RSS - RSS1 - RSS2) / k) / ((RSS1 + RSS2) / (n - 2*k))
    F_critical = stats.f.ppf(1 - alpha, k, n-2*k)
    return F, F_critical, F > F_critical, (a1, b1, a2, b2)

data = np.loadtxt('2.txt', delimiter=',')
x, d1, d2, y = data[:, 0], data[:, 1], data[:, 2], data[:, 3]
print("Тест Чоу для первого качественного признака:")
F1, F_crit1, significant1, coefs1 = chow_test(x, y, d1)
print(f"F-статистика = {F1:.4f}, критическое значение = {F_crit1:.4f}")
print("Тест Чоу для второго качественного признака:")
F2, F_crit2, significant2, coefs2 = chow_test(x, y, d2)
print(f"F-статистика = {F2:.4f}, критическое значение = {F_crit2:.4f}")
```

Запустим скрипт и получим следующие значения F-статистики для первого

качественного признака:

- F-статистика = 1101.6379;
- критическое значение = 3.0420.

Так как значение статистики больше критического, нулевая гипотеза отвергается, а значит имеет смысл разделить выборку по данному качественному признаку.

Значения F-статистики для второго качественного признака:

- F-статистика = 0.3423;
- критическое значение = 3.0420.

Так как значение статистики меньше критического, у нас нет оснований отвергать нулевую гипотезу, а значит нет смысла разделить выборку по данному качественному признаку.

Разобьем выборку по первому качественному признаку и построим модели линейной регрессии. Код для построения модели представлен в Листинге 9.

Листинг 9 — Построение модели линейной регрессии

```
d = d1
x1, y1 = x[d == 0], y[d == 0]
x2, y2 = x[d == 1], y[d == 1]
a1, b1 = linreg_coef(x1, y1)
a2, b2 = linreg_coef(x2, y2)
print(f"Модель для d=0: y = {a1:.4f}x + {b1:.4f}")
print(f"Модель для d=1: y = {a2:.4f}x + {b2:.4f}")

plt.figure(figsize=(10, 6))
plt.scatter(x1, y1, alpha=0.7, label='d = 0')
plt.scatter(x2, y2, alpha=0.7, label='d = 1')
x_line1 = np.linspace(min(x1), max(x1), 100)
x_line2 = np.linspace(min(x2), max(x2), 100)
plt.plot(x_line1, a1*x_line1 + b1, 'r-', label=f'd=0: y = {a1:.4f}x + {b1:.4f}')
plt.plot(x_line2, a2*x_line2 + b2, 'b-', label=f'd=1: y = {a2:.4f}x + {b2:.4f}')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Линейные регрессии по категориям - Файл 2.txt')
plt.legend()
plt.grid()
plt.show()
```

Запустим скрипт и получим следующие значения коэффициентов:

- модель для d=0: $y = 0.0595x + 2.4414$;
- модель для d=1: $y = 0.4115x - 2.4370$.

Графики линейных регрессий представлены на Рисунке 3.

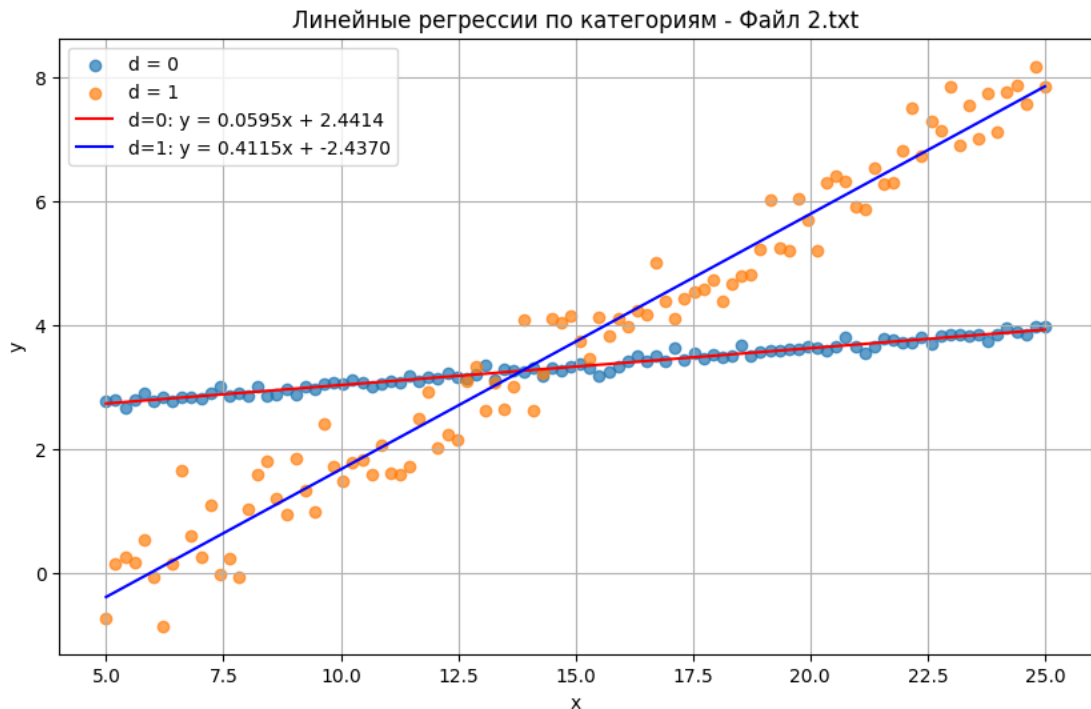


Рисунок 3 — Линейные регрессии по категориям

Как видно из построенного графика, первый качественный признак действительно имеет значение.

1.3 Файл 3

Третий файл содержит 2 ряда данных. С помощью библиотеки `numpy` считаем данные, построим линейную регрессию и оценим значения коэффициентов модели. Код для построения линейной регрессии представлен в Листинге 10.

Листинг 10 — Построение модели линейной регрессии

```
data = np.loadtxt('3.txt', delimiter=',')
x, y = data[:, 0], data[:, 1]
n = len(x)
a, b = linreg_coef(x, y)
print(f"Линейная регрессия: y = {a:.4f}x + {b:.4f}")
```

Запустим скрипт и получим следующую формулу (Формула 2).

$$y = 0.4467x - 12.7937. \quad (2)$$

График модели представлен на Рисунке 4.

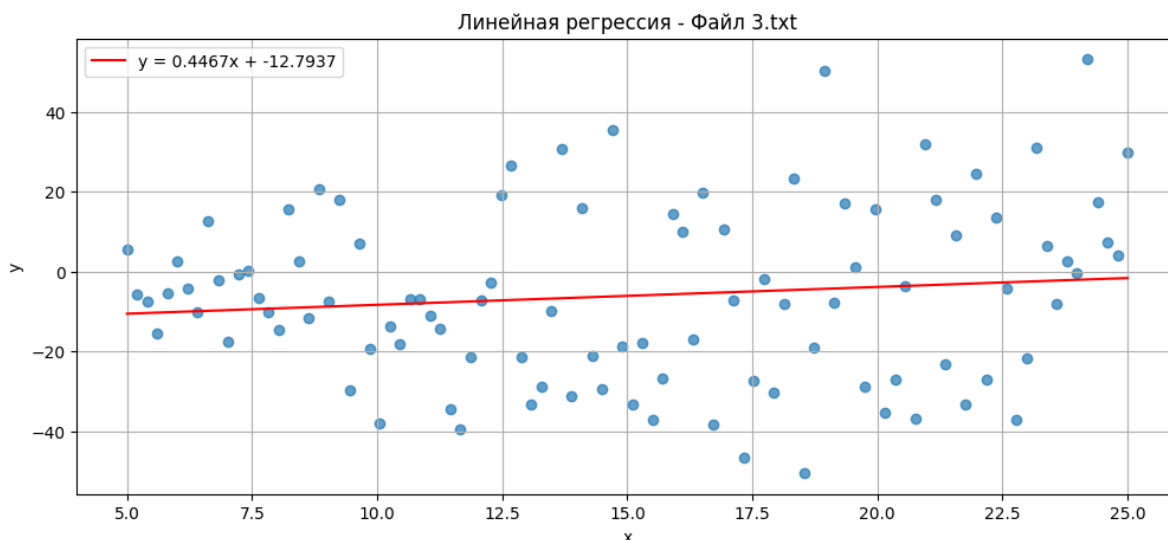


Рисунок 4 — График модели линейной регрессии

После получения коэффициентов регрессии проверим их значимость и адекватность. Код для вычисления представлен в Листинге 11.

Листинг 11 — Проверка значимости и адекватности

```
S_mod = np.sqrt(np.sum((y - (a*x + b))**2) / (n - 2))
T_a, T_b, t_crit, a_sig, b_sig, m_a, m_b = significance_coefs(x, y, a, b, S_mod)
F, F_crit, adequate = f_test_adequacy(x, y, a, b)
print(f"    Коэффициент a: t = {T_a:.4f}, {'значим' if a_sig else 'не значим'}")
print(f"    Коэффициент b: t = {T_b:.4f}, {'значим' if b_sig else 'не значим'}")
print(f"    Модель: F = {F:.4f}, {'адекватна' if adequate else 'не адекватна'})
```

Запустим скрипт и получим, что t-статистика для коэффициента a равна 1.2072, что не является значимой, а для b — -2.1485, что говорит о его значимости.

Значение F-статистики для модели равно 1.0045, что меньше критического (1.3950), значит модель получилась неадекватной.

После установления адекватности модели, определим с помощью тестов Спирмена и Гольдфельда-Квандта наличие в данных гетероскедастичности (Листинг 12).

Листинг 12 — Проверка наличия гетероскедастичности

```
def spearman_test(x, y, a, b):
    residuals = np.abs(y - (a*x + b))
    rank_x = stats.rankdata(x)
    rank_e = stats.rankdata(residuals)
    r_s = 1 - (6 * np.sum((rank_x - rank_e)**2)) / (len(x) * (len(x)**2 - 1))
    t_s = (r_s * np.sqrt(len(x) - 2)) / np.sqrt(1 - r_s**2)
    t_critical = stats.t.ppf(1 - alpha/2, len(x)-2)
    return r_s, t_s, t_critical, abs(t_s) > t_critical

def goldfeld_quandt_test(x, y, a, b):
    sorted_idx = np.argsort(x)
    x_sorted = x[sorted_idx]
    y_sorted = y[sorted_idx]
    n = len(x)
    m = int(3/8 * n)
    x1, y1 = x_sorted[:m], y_sorted[:m]
    a1, b1 = linreg_coef(x1, y1)
    RSS1 = np.sum((y1 - (a1*x1 + b1))**2)
    x2, y2 = x_sorted[-m:], y_sorted[-m:]
    a2, b2 = linreg_coef(x2, y2)
    RSS2 = np.sum((y2 - (a2*x2 + b2))**2)
    F = max(RSS1, RSS2) / min(RSS1, RSS2)
    F_critical = stats.f.ppf(1 - alpha, m-1, m-1)
    return F, F_critical, F > F_critical

r_s, t_s, t_crit_s, hetero_spearman = spearman_test(x, y, a, b)
print(f"Тест Спирмена: r_s = {r_s:.4f}, t = {t_s:.4f}")
print(f"Критическое значение: {t_crit_s}")
print(f"Гетероскедастичность {'присутствует' if hetero_spearman else 'отсутствует'}")
F_gq, F_crit_gq, hetero_gq = goldfeld_quandt_test(x, y, a, b)
print(f"Тест Гольдфельда-Квандта: F = {F_gq:.4f}")
print(f"Критическое значение: {F_crit_gq}")
print(f"Гетероскедастичность {'присутствует' if hetero_gq else 'отсутствует'}")
```

Запустим скрипт и получим следующие значения для теста Спирмена:

- t-статистика = 3.0176;
- критическое значение = 1.9845.

Так как значение t-статистики больше критического, можно сделать вывод о наличии гетероскедастичности в данных.

Тест Гольдфельда-Квандта дал следующие значения:

- F-статистика = 3.5517;
- критическое значение = 1.7430.

Так как значение F-статистики больше критического, можно сделать вывод о наличии гетероскедастичности в данных.

Как видно из проведенного анализа, оба метода подтвердили наличие гетероскедастичности в данных.

ВЫВОД

В ходе выполнения практической работы были освоены навыки построения линейных регрессионных моделей, оценку их адекватности, проверку значимости параметров, анализ гетероскедастичности и применение теста Чоу для сравнения моделей на разных подвыборках.